Adevar Labs

Prepared for:
Spiko

June 12, 2026

Solana Minter Program Audit Report

Version: after fix review

Table of Contents

1. Executive Summary

About Spiko's Minter Program

Spiko is an organization issuing RWA assets on-chain, currently operating on five EVM-compatible blockchains, as well as Stellar and Starknet.

The Minter program reviewed in this report is the Solana integration of this product.

Audit Summary

The table below summarizes identified vulnerabilities by risk level and remediation status.

Risk Level	Count	Fixed	Acknowledged
Critical	0	0	0
High	0	0	0
Medium	0	0	0
Low	2	2	0

Enhancement opportunities: 3

Overall Security Posture

The following sections describe the security posture before and after the fix review. The fix review is the stage of the audit where Adevar Labs checked the fixes implemented by the Spiko development team for the issues found during the audit.

The Spiko team demonstrated a good understanding of the Solana Anchor framework, as well as Token-2022 and ACL extension model. As Spiko is a regulated financial entity in the EU, the Solana program features a high degree of centralization by design. The use of a Squads multisig to govern privileged operations such as program upgrades and admin key management is a strong security practice that meaningfully reduces single-point-of-failure risk if implemented correctly.

After Fix Review Summary

The fix review confirmed successful resolution of both Low severity findings. The Spiko team also implemented the most important enhancement opportunity preventing unexpected loss of admin control or mint capability during authority transfer.

Before Fix Review Summary

The audit identified 2 Low severity issues and 3 enhancement opportunities. The first Low severity issue concerns a missing vault PDA validation in the **setup-minter.ts** deployment script, where passing the raw multisig account address instead of the derived vault PDA renders all admin instructions permanently unexecutable with no recovery path. The second Low severity issue relates to the **set_daily_limit** instruction unconditionally resetting **used_amount** to zero on both initialization and updates, allowing the full daily limit to be reused the same day whenever the limit is adjusted.

Some enhancement opportunities were also identified covering hardening governance of the Minter upgrade authority and admin key through proper multisig thresholds and hardware wallet practices; adding two-step authority transfer (nominate then transfer) flows to **set_admin** and **set_mint_authority** to prevent irrecoverable lockout from a single mistyped pubkey; and binding the **mint** account in **SetMintAuthority** to a program-managed token to eliminate the operator footgun of transferring authority on the wrong mint.

Launch Recommendations

After Fix Review Summary

All strongly recommended recommendations have been addressed by the team.

Before Fix Review Summary

I. Strongly recommended:

- Address the **set_daily_limit** counter reset: separate the initialization path from the update path so that **used_amount** and **last_day** are preserved on updates, preventing circumvention of the daily mint cap through a mid-day limit change.
- Address the **setup-minter.ts** vault PDA mismatch: derive the Squads vault PDA from the multisig address before calling **set_admin** and assert equality, since setting the wrong address produces an unrecoverable state where no admin operation can ever be executed.
- Implement the two-step authority transfer pattern for **set_admin** and **set_mint_authority** to prevent permanent loss of admin control or mint capability from a single mistyped public key.
- Consider transferring the program upgrade authority from the deployer keypair to a Squads multisig immediately after deployment, and set **minter_config.admin** to the vault PDA rather than the multisig account address, validating this at deploy time.
- Adopt the highest practical approval threshold for the upgrade authority and the **approve_mint**-capable admin key, with each signer on a dedicated hardware wallet and no single device or person able to reach quorum alone.

II. Post-Launch Monitoring:

- Enable alerting on all Squads multisig events: proposed transactions, approvals, rejections, and any signer or threshold changes for each multisig vault involved in upgrade authority and admin operations.
- Monitor on-chain **approve_mint** and **set_daily_limit** transactions and reconcile minted amounts against **used_amount** in **MintDailyLimit** daily to detect any unexpected resets or overflows.
- Have signers verify decoded Squads instruction data against the **printInstruction** output before approving any proposal that calls **approve_mint**, **set_daily_limit**, or **setAuthority**, to guard against malicious proposal substitution.
- Periodically verify that the program upgrade authority remains set to the expected Squads multisig vault PDA and has not been silently transferred.
- Maintain an off-chain registry of all Token-2022 authorities (permanent delegate, pause, mint close, freeze, metadata update, and ABL gate authority) with documented owners, thresholds, and rotation procedures, and audit this registry after any deployment or configuration change.

Audit Scope

- **Repository:** https://github.com/spiko-tech/solana-contracts
- **Before-Fix Review Commit Hash:** **2383424a630694988b2c160e6a1aaf828f5565a9**
- **After-Fix Review Commit Hash:** **cb542f9a9ba2e588762cc8aa0bed4dc842adbe06**
- **Files/Modules in Scope:**
  - **programs/minter/src**
  - **scripts**

2. Assumptions and Considerations

Project & Protocol Assumptions

- Privileged pubkeys passed to **set_admin**, **set_mint_initiator**, and **set_mint_authority** are valid and controllable.
- Privileged roles (the Squads multisig admin, the mint initiator, and the token/extension authority vaults) are trusted to be kept secure.
- Keypair files supplied to the scripts are trusted, well-formed, and kept secure.
- Allow-list and block-list membership is curated correctly off-chain to reflect KYC and sanctions status.

Centralization

- The deployer EOA retains the program upgrade authority after divestment and can replace the entire program logic, overriding all on-chain rules.
- The Minter admin multisig can approve queued mints with no daily-limit check, set or raise the daily limit, rotate the admin and mint initiator, and move the SPL mint authority off the program, all without a timelock.
- The mint initiator hot key can mint autonomously up to the daily limit.
- The Permanent Delegate (immutable) can claw back or move any holder's tokens at any time.
- The Pause Authority can halt all token transfers globally via the Token-2022 pausable extension.

3. Severity Definitions

Each issue identified in this report is assigned a severity level based on two dimensions: **Impact** and **Likelihood**. These dimensions help project our team's understanding of both the potential consequences of a vulnerability and how likely a vulnerability is to be discovered and exploited in the real world.

Note: Enhancements represent non-blocking improvements–typically usability, observability, or defense-in-depth tweaks that do not pose an immediate asset risk but would improve the product's reliability and/or user experience if implemented.

Impact

Impact reflects the potential consequences of the issue–particularly on **project funds**, **user funds**, and the **availability or integrity** of the protocol.

- **High Impact**: Successful exploitation could result in a complete loss of user or protocol funds, disruption of core protocol functionality, or permanent loss of control over critical components.
- **Medium Impact**: Exploitation could cause significant disruption or partial loss of funds, but not a total compromise. May impact some users or non-core functionality.
- **Low Impact**: The issue has minor or negligible consequences. It may affect edge cases, expose metadata, or degrade performance slightly without putting funds or core logic at serious risk.

Likelihood

Likelihood reflects how easy a vulnerability is to discover and exploit by an attacker, as well as how economically attractive the exploit is to an attacker.

- **High Likelihood**: The vulnerability is trivially exploitable. This means it can be exploited by a wide range of actors without privileged access rights, with minimal capital requirements and low financial risks.
- **Medium Likelihood**: This type of vulnerability can be found and exploited with moderate effort. It might require a significant capital investment, but with manageable financial risk.
- **Low Likelihood**: Exploitation of these vulnerabilities is often technically unfeasible or requires highly specialized conditions. They may require extraordinary effort or a significant financial risk for an attacker, with a high chance of failure and minimal potential return.

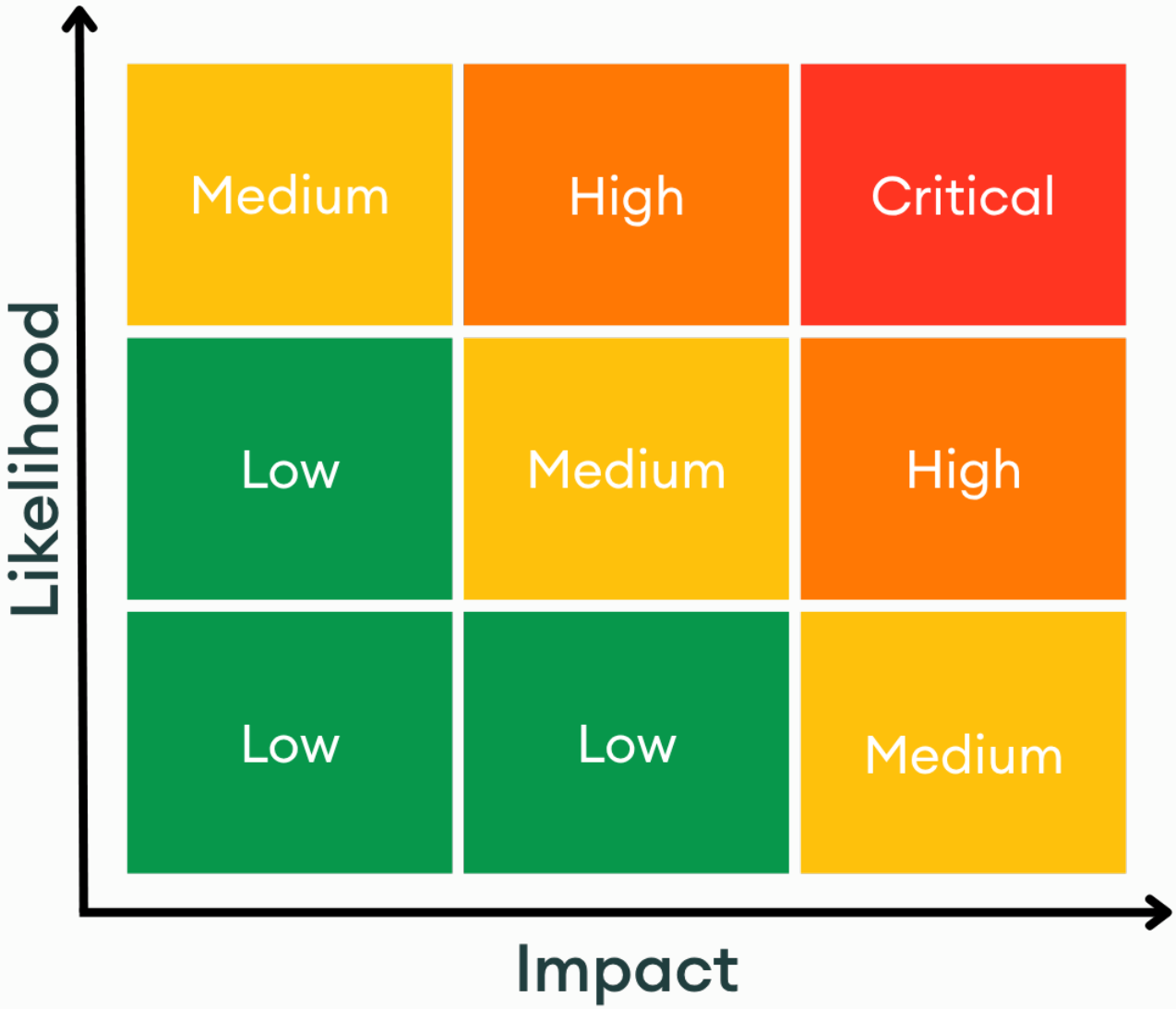
Severity Classification Matrix

By combining **Impact** and **Likelihood**, we assign a severity level using the matrix below:

- **Critical**: High impact + high likelihood (e.g. a bug that could allow anyone to drain a substantial amount of protocol funds with minimal effort)
- **High**: High impact with medium likelihood, or medium impact with high likelihood
- **Medium**: Moderate impact and/or discoverability

- **Low**: Minimal impact or unlikely to be exploited

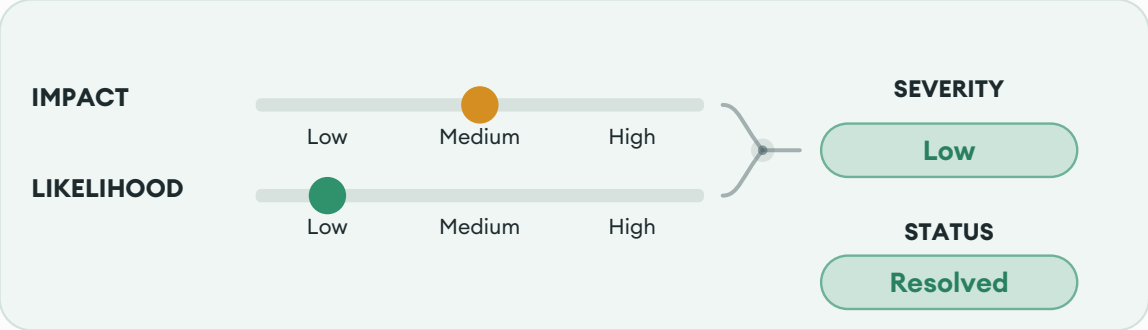
This structured approach helps teams prioritize fixes and mitigate the most dangerous threats first.



Severity Matrix

4. Findings

L01: setup-minter.ts sets minter_config.admin without verifying it is the Squads vault PDA



LOCATION

solana-contracts/2383424a/scripts/setup-minter.ts:L63-L65

RELEVANT SNIPPET

>_ setup-minter.ts — TYPESCRIPT

```
61:
62: console.log(`\n-- Transferring config admin to multisig... --`);
63: const setAdminIx = await getSetAdminInstructionAsync({
64:   admin,
65:   newAdmin: address(minterAdmin),
66: });
67: await sendTx(rpc, rpcSub, admin, [setAdminIx], `Set admin → ${minterAdmin}`);
```

DESCRIPTION

initializeAndTransferAdmin calls **getSetAdminInstructionAsync({ admin, newAdmin: address(minterAdmin) })**, writing the raw **--minter-admin** value into **minter_config.admin** after only a base58 check.

Every admin instruction (**set_daily_limit**, **set_admin**, **set_mint_initiator**, **set_mint_authority**, **approve_mint**, **cancel_mint**) enforces **minter_config.admin == admin.key()**, and these are executed through Squads, where the signer of a multisig's inner instructions is the vault PDA.

If **--minter-admin** is set to anything other than that vault PDA, such as the multisig account address that the usage string **<multisig-pubkey>** invites, every admin proposal fails the constraint.

The state is unrecoverable because **set_admin** is itself admin-gated, so only the unsignable stored admin could change it, and the deployer already handed off admin in the same function.

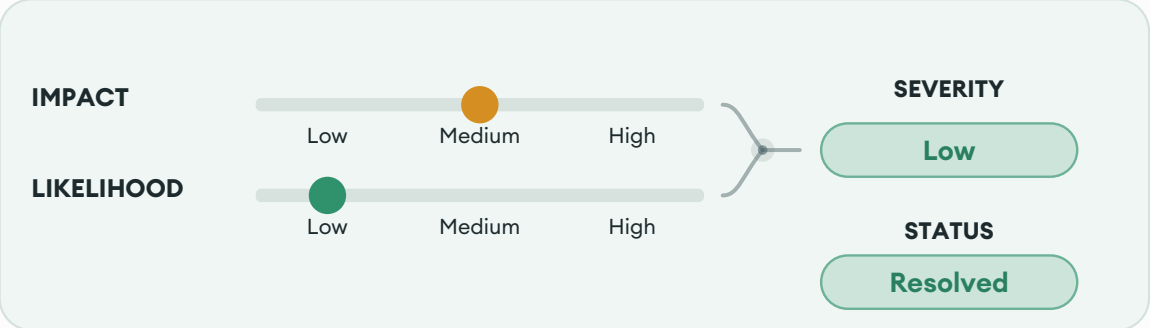
RECOMMENDATION

Derive the vault PDA and assert it matches before sending **set_admin**.

FIX REVIEW NOTES

Fixed according to the recommendation.
The script no longer accepts a single ambiguous **--minter-admin** argument. It now requires **--minter-admin-squad-account** and **--minter-admin-squad-vault**, derives vault index 0 with **multisig.getVaultPda**, and aborts if the provided vault does not match the derived PDA.

L02: used_amount is set to zero on every limit change regardless of current day



LOCATION

solana-contracts/2383424a/programs/minter/src/instructions/set_daily_limit.rs:L36-L50

RELEVANT SNIPPET

>_ set_daily_limit.rs — RUST

```rust
}

pub(crate) fn handler(ctx: Context<SetDailyLimit>, limit: u64) -> Result<()> {
    let clock = Clock::get()?;
    let current_day = clock
        .unix_timestamp
        .checked_div(SECONDS_PER_DAY)
        .ok_or(MinterError::ArithmeticOverflow)?;

    let daily_limit = &mut ctx.accounts.mint_daily_limit;
    daily_limit.limit = limit;
    daily_limit.used_amount = 0;
    daily_limit.last_day = current_day;
    daily_limit.bump = ctx.bumps.mint_daily_limit;

    Ok(())
}
```

DESCRIPTION

The **set_daily_limit** instruction always sets **used_amount = 0** whenever it is called, regardless of whether the account is being initialized for the first time or having its limit updated.

The account is created with **init_if_needed**, meaning the same code path runs for both first-time initialization and subsequent updates. For initialization, setting **used_amount** to zero is correct. For updates, it is not: any accumulated usage for the current day is discarded.

An admin who legitimately updates the limit mid-day (for example, raising or lowering it in response to operational needs) will also reset the **used_amount** counter. This means tokens already minted that day are no longer counted, and the full new limit becomes available again immediately.

RECOMMENDATION

Separate the initialization path from the update path. On update, only modify limit and leave **used_amount** and **last_day** untouched.

FIX REVIEW NOTES

Fixed according to the recommendation.
set_daily_limit now only clears usage when the computed day changes.

5. Enhancement Opportunities

E01: Harden governance of the Minter upgrade authority and admin key

LOCATION

solana-contracts/2383424a/scripts/lib/executeSquadsProposal.ts:L16-L23

RELEVANT SNIPPET

>_ executeSquadsProposal.ts TYPESCRIPT

```
14: }
15:
16: export const executeSquadsProposal = async (opts: {
17:   rpcUrl: string;
18:   keypairPath: string;
19:   multisigPubkey: string;
20:   vaultIndex: number;
21:   instructions: KitInstruction[];
22:   label?: string;
23: }) => {
24:   const { rpcUrl, keypairPath, multisigPubkey, vaultIndex, instructions, label } = opt
      s;
25:   const connection = new Connection(rpcUrl, 'confirmed');
```

DESCRIPTION

The protocol concentrates several authorities whose power exceeds any on-chain control: the Minter program upgrade authority (left on the single deployer keypair), the Minter admin (which can mint unbounded supply via **approve_mint**, outside the **MintDailyLimit** accounting), and the token-level authorities (permanent delegate, pause, mint close, freeze via the Token ACL config, metadata update, and the ABL gate authority that owns the allow and block lists).

These are configured by the deploy scripts but have no documented classification, threshold policy, signer requirements, or monitoring. Compromise or misconfiguration of any of these allows unlimited minting, holder seizure, freezing, or full program replacement.

POTENTIAL BENEFIT

Removes the single-key and single-quorum paths to unlimited minting, asset seizure, and arbitrary program upgrades.

RECOMMENDATION

- Transfer the program upgrade authority from the deployer keypair to a Squads multisig after deployment.
- Treat the upgrade authority and the **approve_mint**-capable admin as the protocol's two most powerful keys and govern them with the highest threshold the team adopts.
- Set **minter_config.admin** to the Squads vault PDA, not the multisig account address, and validate this at deploy time.
- Have signers verify the decoded Squads instruction against the **printInstruction** output before approving **approve_mint**, **set_daily_limit**, and **setAuthority** proposals.

General multisig best practices:
- Each signer uses a dedicated hardware wallet; private keys are never stored on an internet-connected or shared machine.
- No single device or person holds enough keys to meet the threshold, and signers are kept independent so that one compromised machine cannot reach quorum.
- Enable alerting on proposed transactions, approvals, and signer or threshold changes for each multisig.

DEVELOPER RESPONSE

Acknowledged. The remediation for this finding falls outside the scope of the current engagement and will be addressed in a subsequent phase.

E02: Single-step authority transfers allow irrecoverable lockout in set_admin and set_mint_authority

LOCATION

solana-contracts/2383424a/programs/minter/src/instructions/set_admin.rs:L20-L22

RELEVANT SNIPPET

>_ set_admin.rs RUST

```rust
}

pub(crate) fn handler(ctx: Context<SetAdmin>, new_admin: Pubkey) -> Result<()> {
    ctx.accounts.minter_config.admin = new_admin;
    Ok(())
}
```

DESCRIPTION

Authority transfers write the new authority immediately with no nomination and acceptance step and no **Pubkey::default()** validation.

A wrong or zero key in **set_admin** overwrites **minter_config.admin** and locks every admin instruction, including **set_admin** itself, with no recovery path.

A wrong key in **set_mint_authority** moves **MintTokens** authority off the **MinterConfig** PDA via the **set_authority** CPI, after which the program can never mint that token again.

POTENTIAL BENEFIT

Eliminates permanent loss of admin control and mint capability from a single mistyped pubkey.

RECOMMENDATION

Add a two-step ownership transfer for **set_admin** and **set_mint_authority**, where the new authority must call an **accept_*** instruction to finalize the ownership transfer.

FIX REVIEW NOTES

Fixed according to the recommendation.
A mistyped or zero key no longer moves authority immediately; the current admin can cancel the pending nomination before acceptance.

E03: Bind mint in SetMintAuthority to a program-managed token

LOCATION

solana-contracts/2383424a/programs/minter/src/instructions/set_mint_authority.rs:L21

RELEVANT SNIPPET

>_ set_mint_authority.rs RUST

```
19:
20: #[account(mut)]
21: pub mint: InterfaceAccount<'info, Mint>,
22:
23: pub token_program: Interface<'info, TokenInterface>,
```

DESCRIPTION

The **mint** account in **SetMintAuthority** is constrained only by **#[account(mut)]**, with no **seeds**, **address**, or **constraint** binding it to a token this program manages.

The **set_authority** CPI signed by the **MinterConfig** PDA fails unless the PDA is the current mint authority, so abuse is limited to tokens the program already controls.

If the **MinterConfig** PDA becomes mint authority of more than one token over time, the instruction cannot distinguish them, so an admin can transfer **MintTokens** authority on the wrong mint by mistake.

POTENTIAL BENEFIT

Removes the operator footgun by making it impossible to pass a mint the program does not manage.

RECOMMENDATION

Bind **mint** to a known token.

Depending on the design the Spiko team wants to proceed with:

- For a multi-token design, require the per-mint **MintDailyLimit** PDA (consistent with **initiate_mint**).
- For a single-token design, store the mint in MinterConfig and enforce address = minter_config.mint on mint.

DEVELOPER RESPONSE

Acknowledged by the team as it did not involve any risk.

About Us

About Adevar Labs

Adevar Labs is a boutique blockchain security firm specializing in web3 audits.

Built by a mix of experienced professionals in traditional enterprise and crypto natives who have contributed to some of the most critical projects in blockchain infrastructure.

Our team's background spans companies like Bitdefender, Asymmetric Research, Quantstamp, Chainproof, and Juicebox, and includes experience securing smart contracts, bridges, and L1 and L2 protocols across ecosystems like Solana, Ethereum, Polkadot, Cosmos, and MultiversX.

With over 100 audits completed and a portfolio that includes custom fuzzers, exploit modeling, and runtime testing frameworks, Adevar Labs brings both depth and precision to every engagement.

Our auditors have discovered critical vulnerabilities, built high-impact tooling, and placed in top positions in premier audit competitions including Code4rena and Sherlock.

Team members hold distinctions such as PhDs in software protection, and key roles at Fortune 500 companies, and leadership of flagship conferences like ETH Bucharest.

With team members having publications with over 1,100 academic citations and trusted by projects securing over $500M in on-chain value, Adevar Labs blends elite technical rigor with real-world security impact.

We also collaborate with some of the best independent security researchers in the web3 space.

Projects may optionally request specific contributors to be part of their audit.

Audit Methodology

Our audit methodology is specialized to provide thorough security assessments of Solana programs. We focus explicitly on rigorous manual analysis, detailed threat modeling, and careful validation of implemented fixes to ensure your programs operate securely and as intended.

1. Program Context and Architecture Analysis

Our auditors begin by deeply examining your Solana program's documentation, intended functionality, and account design. We meticulously map out program interactions, instruction processing flows, and state management logic. Special attention is given to understanding how your program interfaces with critical Solana system programs, such as the SPL Token Program, Stake Program, and System Program. Last but not least we check external integrations with other Solana projects and verify if the inputs and return values are handled properly.

2. Threat Modeling

We conduct targeted threat modeling tailored specifically for Solana's execution environment. We carefully define attacker capabilities and identify potential vulnerabilities that may arise from Solana-specific issues, including but not limited to:

- Unauthorized account data manipulation
- Improper ownership or signer verification
- Misuse of Program-Derived Addresses (PDAs)
- Incorrect use of Cross-Program Invocations (CPI)
- Failure to adequately handle account privileges or account states
- Risks stemming from rent-exemption and account initialization logic

3. In-depth Manual Security Review

Our experienced auditors perform an extensive manual security review of your Rust-based Solana programs. This involves a comprehensive line-by-line inspection of source code, focusing on common Solana vulnerabilities including, but not limited to:

- Missing or insufficient ownership checks
- Inadequate signer checks
- Incorrect handling of CPI calls and invocation privileges
- Arithmetic and integer overflow or underflow errors
- Unsafe deserialization and serialization of account data structures
- Improper token transfers and SPL-token logic issues
- Logic flaws in financial operations or state transitions
- Edge-case handling in instruction input validation
- Potential denial-of-service vectors related to transaction execution and account handling

During this stage, we clearly document any discovered vulnerabilities, including detailed descriptions, precise severity ratings, and recommendations for secure implementation. In situations where it is not clear how the vulnerability might be exploited we may also include a detailed proof-of-concept exploit including code snippets and instructions on how the exploit could be performed.

4. Detailed Fix Review and Validation

After the initial audit and your team's subsequent remediation efforts, we perform a comprehensive fix review to ensure the vulnerabilities identified have been effectively resolved.

We verify each fix individually, confirming that:

- Corrections effectively eliminate the security risks
- Changes do not inadvertently introduce new vulnerabilities or regressions
- The fixes align closely with Solana best practices and secure coding guidelines

Our detailed validation ensures that security improvements are robust, complete, and aligned with best practices specific to the Solana development ecosystem.

Confidentiality Notice

This report, including its content, data, and underlying methodologies, is subject to the confidentiality and feedback provisions in your agreement with Adevar Labs. These materials are not to be disclosed, extracted, copied, or distributed except to the extent expressly authorized by Adevar Labs.

Legal Disclaimer

The review and this report are provided by Adevar Labs on an as-is, where-is, and as-available basis. To the fullest extent permitted by law, Adevar Labs disclaims all warranties, expressed or implied, in connection with this report, its content, and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement.

You agree that access to and/or use of the report and other results of the review, including any associated services, products, protocols, platforms, content, and materials, will be at your sole risk.

FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE. This report is based on the scope of materials and documentation provided for a limited review at the time provided.

You acknowledge that blockchain technology remains under development and is subject to unknown risks and flaws. Adevar Labs does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party, or any open-source or third-party software, code, libraries, materials, or information accessible through the report. As with the purchase or use of a product or service in any environment, you should use your best judgment and exercise caution where appropriate.